

به نام خداوند جان و خرد

پیش‌گزارش آزمایش اول

آزمایشگاه طراحی سیستم‌های دیجیتال

اعضای گروه

پوریا رحمانی ۴۰۲۱۱۱۴۱۸

امیرمحمد شربتی ۴۰۲۱۰۶۱۱۲

موضوع آزمایش

الف) طراحی یک مدار به صورت شماتیک در Quartus بطوریکه ورودی آن یک عدد چهاررقمی BCD است و خروجی آن یک بیت است که برابر 1 است اگر و فقط اگر عدد BCD ورودی بر 3 بخش‌پذیر باشد.

ب) همان بخش الف، فقط خروجی 1 است اگر و فقط اگر عدد ورودی بر 11 بخش‌پذیر باشد.

تئوری پشت مدار

الف) فرض کنید عدد BCD ما به صورت $a_3a_2a_1a_0$ باشد که هر کدام از a_i ها یک عدد چهار بیتی هستند که به عنوان ورودی به ما داده می‌شوند. می‌دانیم این عدد بر 3 بخش‌پذیر است اگر و فقط اگر حاصل جمع $a_3 + a_2 + a_1 + a_0$ بر 3 بخش‌پذیر باشد. بنابراین ابتدا، این حاصل جمع را محاسبه می‌کنیم:

ابتدا حاصل جمع‌های $a_3 + a_1$ و $a_2 + a_0$ را محاسبه می‌کنیم. هر دوی این اعداد را می‌توان با نمایشی 5 بیتی نمایش داد. حال این دو حاصل جمع را نیز با هم جمع می‌کنیم تا حاصل جمع هر چهار عدد بدست آید. جمع دو عدد 5 بیتی نمایشی 6 بیتی دارد. پس حاصل جمع این چهار عدد را می‌توان در 6 بیت نمایش داد. حالا باید ببینیم عدد باینری 6 بیتی حاصل بر 3 بخش‌پذیر هست یا خیر. فرض کنید عدد به صورت $b_5b_4b_3b_2b_1b_0 \equiv (b_0 + b_2 + b_4) + 2(b_1 + b_3 + b_5) \text{ mode } 3$ باشد. کافیت اعداد $b_0 + b_2 + b_4$ و $b_1 + b_3 + b_5$ و سپس عدد سمت راست همنهشتی را محاسبه کنیم. این عدد حداکثر برابر 9 خواهد بود و در نتیجه نمایشی چهار بیتی خواهد داشت. حال اگر این عدد برابر 0، 3، 6 یا 9 باشد، عدد اصلی بر 3 بخش‌پذیر بوده است و اگر نه، بر 3 بخش‌پذیر نبوده است. براساس همین روش مدار خود را خواهیم ساخت.

ب) باز هم فرض کنید عدد BCD ما به صورت $a_3a_2a_1a_0$ باشد (با تعریف مشابه بخش پیش). می‌دانیم عدد بر 11 بخش‌پذیر است اگر و فقط اگر حاصل $(a_3 + a_1) - (a_2 + a_0)$ بر 11 بخش‌پذیر باشد. بنابراین، مشابه بخش پیش، اعداد $a_3 + a_1$ و $a_2 + a_0$ را محاسبه می‌کنیم. سپس مکمل دو $a_2 + a_0$

را محاسبه کرده با $a_3 + a_1$ جمع می‌کنیم. توجه کنید حاصل $a_0 + a_2$ و $a_3 + a_1$ را در 5 بیت به صورت بی‌علامت نشان داده بودیم و در نتیجه برای نمایش مکمل دو به حداقل 6 بیت نیاز داریم. (برای مکمل دو گرفتن از $a_0 + a_2$ ، یک 1 به سمت چپ آن اضافه می‌کنیم، پنج بیت نمایش بی‌علامت آن را not می‌کنیم و یک واحد به آن اضافه می‌کنیم) (البته یک واحد را در carry in هنگام محاسبه $(a_3 + a_1) - (a_2 + a_0)$ لحاظ می‌کنیم)

حاصل این تفریق که به دلیل ignore کردن carry out در جمع علامت دار، باز هم 6 بیتی خواهد بود، در بازه ± 18 است (چون حاصل تفریق دو عدد نامنفی کمتر یا مساوی 18 است) و بنابراین بر 11 بخش‌پذیر خواهد بود اگر و فقط اگر یکی از اعداد 0 یا ± 11 باشد. بر اساس همین منطق، مدار خود را خواهیم ساخت.

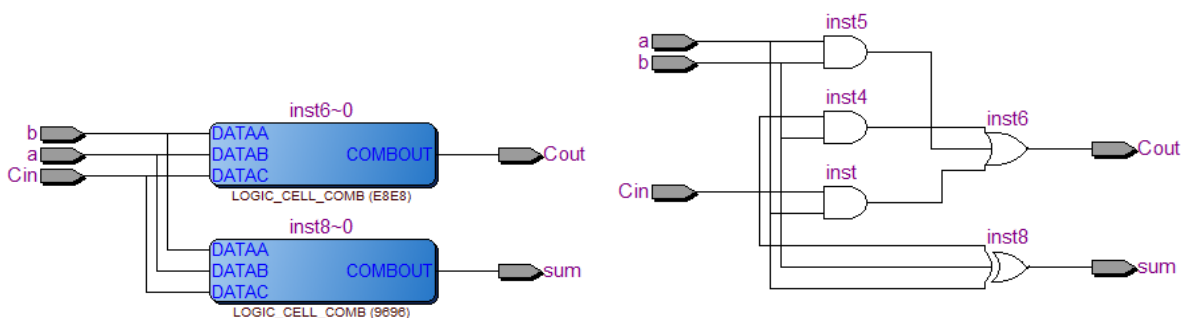
طراحی مدار

توجه کنید در مدار تنها و تنها از گیت‌های منطقی استفاده شده است اما برای راحتی رسم شماتیک، طراحی، به صورتی که توضیح خواهیم داد، ماژول به ماژول انجام شده است و در مدار اصلی از ماژول‌هایی که خودمان طراحی کردیم استفاده می‌کنیم.

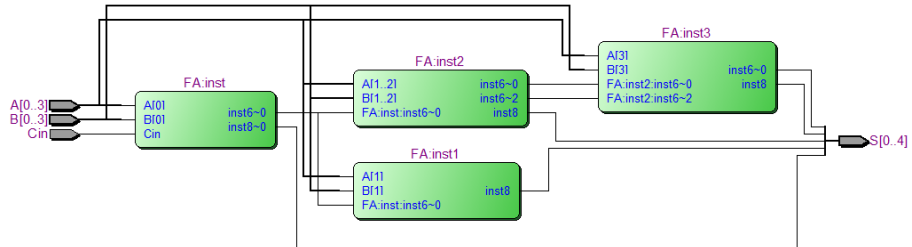
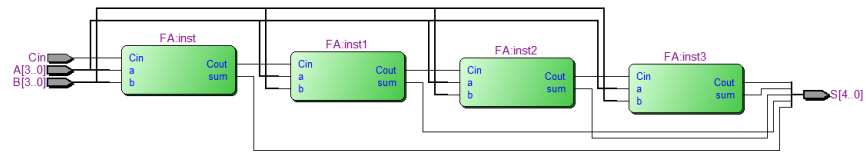
الف) بخش‌پذیری بر 3

ماژول Full adder: اولین ماژولی که طراحی کردیم، ماژول Full adder است که مطابق معمول طراحی شده است $(sum = a \oplus b \oplus Cin)$ و $(cout = ab + aCin + bCin)$

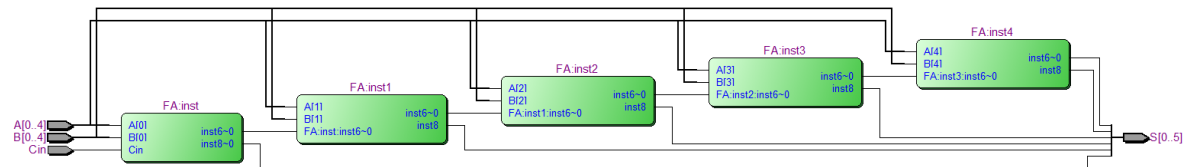
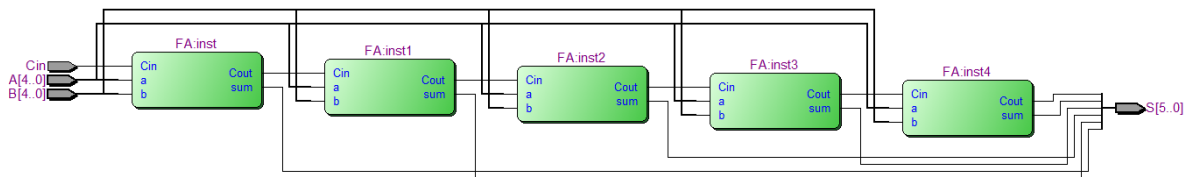
در زیر تصاویر RTL و PostMapping technology mapping این ماژول را می‌بینید:



ماژول جمع‌کننده چهاربیتی: برای محاسبه حاصل جمع $a_0 + a_2$ و $a_1 + a_3$ ، با استفاده از ماژول Fulladder خود طراحی کرده بودیم، یک ripple carry adder چهاربیتی ساختیم که تصاویر RTL و technology mapping آن را در صفحه بعد می‌بینید:

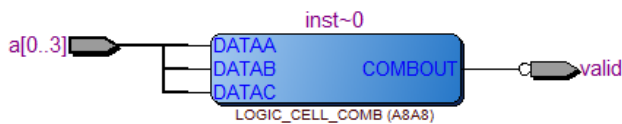


ماژول جمع کننده 5 بیتی: برای محاسبه حاصل جمع دو عدد $a_0 + a_2$ و $a_1 + a_3$ یک ripple carry adder پنج بیتی نیز ساختیم:

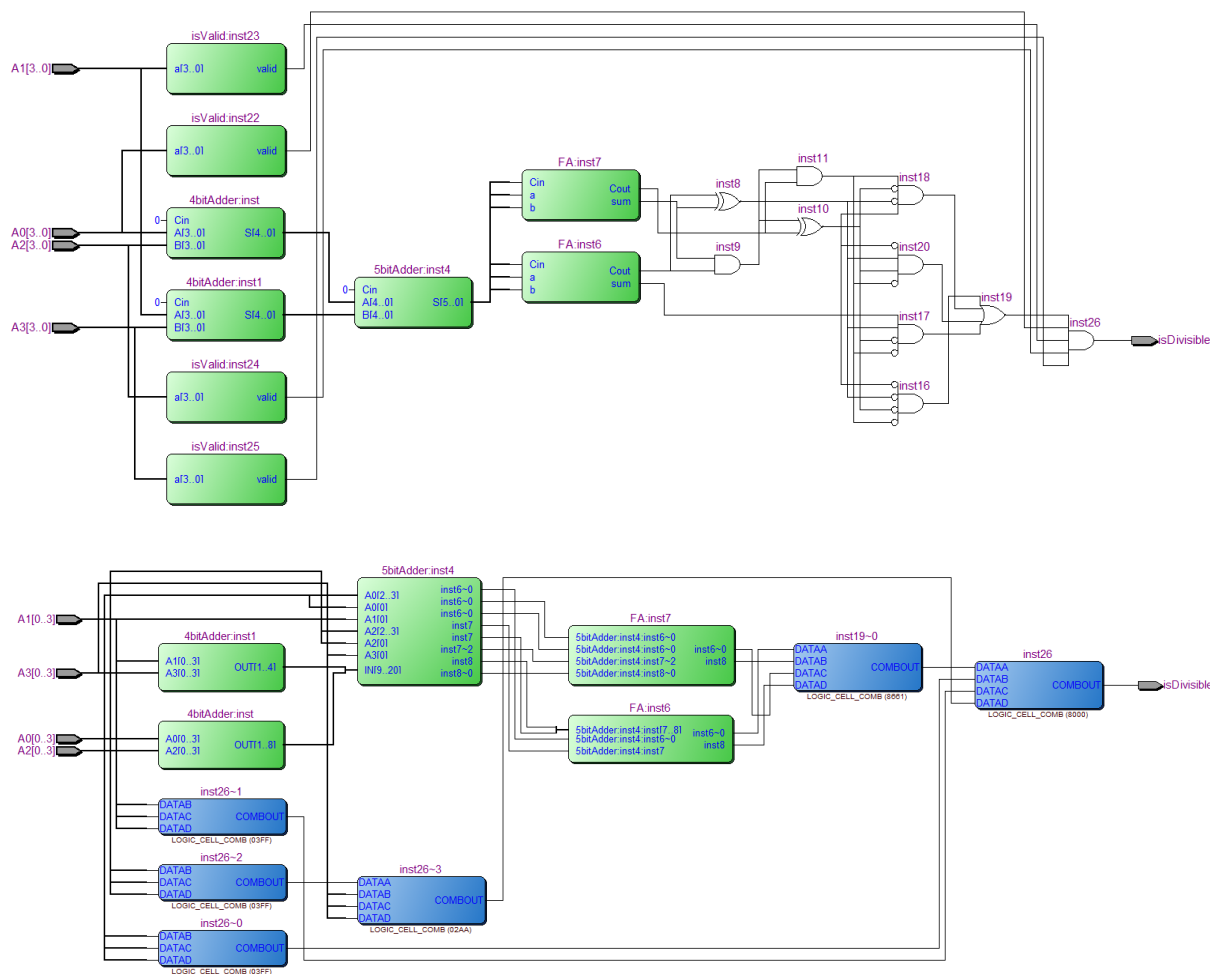


ماژول **is_valid**: این ماژول تنها برای این تعبیه شده است که بررسی کند آیا رقم‌های ورودی واقعا رقم‌های valid (کمتر یا مساوی 9 هستند یا خیر)، به این صورت که یک عدد چهاربیتی $b_3b_2b_1b_0$ از نه بیشتر است اگر و فقط اگر

$$(b_3 == 1) \&\& (b_2 | b_1 == 1)$$

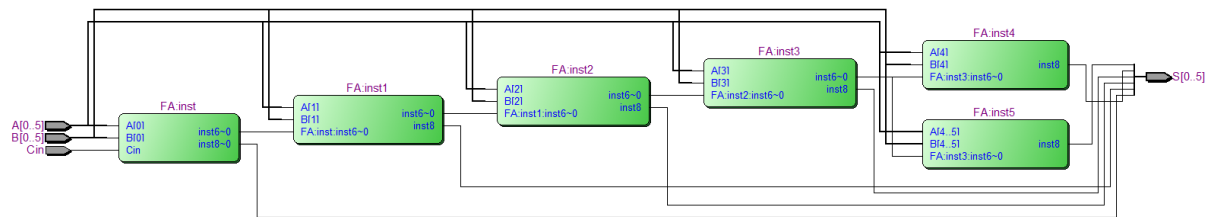
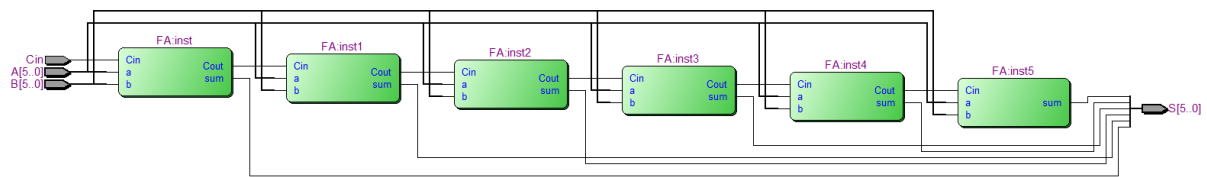


در نهایت با استفاده از ماژول‌هایی که ساختیم و بر اساس منطقی که توضیح دادیم، کل مدار را تشکیل دادیم:

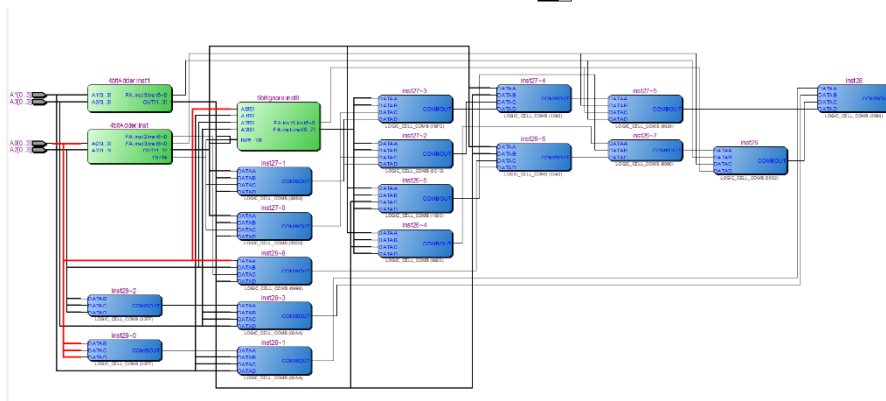
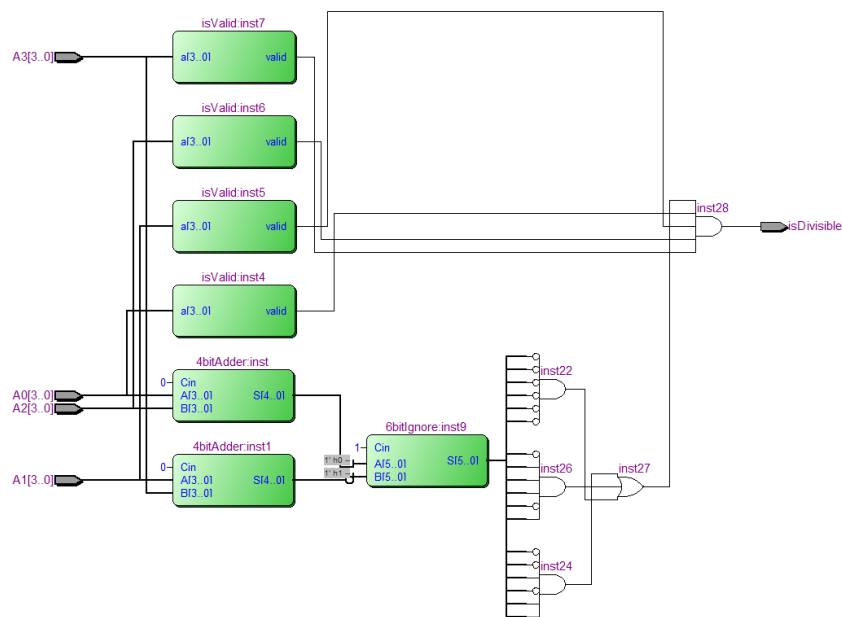


ب) بخش پذیری بر 11

مشابه حالت بخش پذیری بر 3، به verify کردن درستی ورودی‌ها نیاز داریم و بنابراین ماژول `is_valid` را عیناً مشابه آنچه برای حالت بخش پذیری بر 3 طراحی کردیم قرار می‌دهیم. همچنین به جمع کردن $a_0 + a_2$ و $a_1 + a_3$ نیاز داریم. بنابراین مشابه بخش پیش باید ماژول Full adder و ماژول ripple carry adder چهار بیتی را داشته باشیم و مشابه بخش پیش از آن‌ها استفاده کنیم. برای تفریق دو عدد پنج بیتی بی‌علامت حاصل که آن‌ها را به نمایش شش بیتی علامت‌دارشان تبدیل کرده‌ایم (مطابق توضیحی که در بخش تئوری گفتیم) نیاز به یک جمع کننده شش بیتی با `Cin` (باید بتوانیم `Cin` را یک بدهیم تا جای بعلاوه یک در محاسبه مکمل دو را پر کند) داریم که در خروجی `carry out` را در نظر نمی‌گیرد. بنابراین این ماژول را نیز به صورت ripple carry طراحی کردیم:



و در نهایت نیز مدار را مطابق آنچه در بخش تئوری پشت مدار گفتیم، طراحی کردیم.



تصاویر شکل موج نتایج تست

در اینجا نتایج چند تست را نشان می‌دهیم (توجه کنید برای راحتی، مقدار رقم‌های BCD در مبنای 10 نمایش داده شده‌اند):

الف) بخش‌پذیری بر 3

تصویر اول یک عدد BCD که بر 3 بخش‌پذیر است را نشان می‌دهند. تصویر دوم یک عدد BCD که بر 3 بخش‌پذیر نیست و تصویر آخر چهار عدد چهار بیتی را نشان می‌دهد که مجموعشان بر 3 بخش‌پذیر است اما از آنجا که یکی از آن‌ها بزرگتر مساوی 10 است، خروجی isDivisible یک نمی‌شود:



ب) بخش‌پذیری بر 11

تصویرهای اول، دوم و سوم به ترتیب اعداد BCD را نشان می‌دهند که در آن‌ها $(a_0 + a_2) - (a_1 + a_3)$ برابر صفر، 11 و منفی یازده است. تصویر چهارم چهار عدد چهار بیتی را نشان می‌دهد که حاصل $(a_0 + a_2) - (a_1 + a_3)$ در آن بر 11 بخش‌پذیر است اما از آنجا که یکی از اعداد بزرگتر مساوی 10 است، سیگنال isDivisible برابر صفر شده‌است. تصویر آخر نیز یک عدد BCD را نشان می‌دهد که بر 11 بخش‌پذیر نیست.

